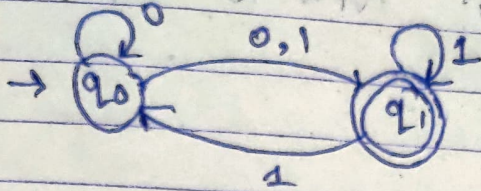


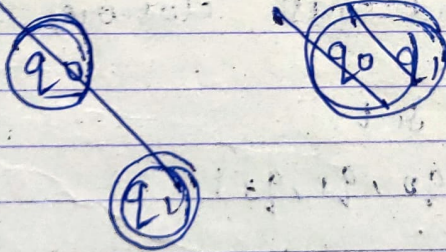
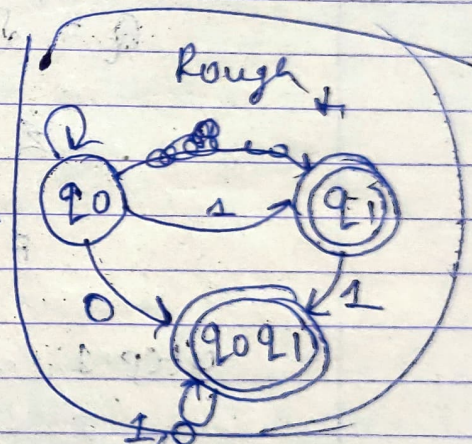
Q8 NFA TO DFA

T.T OF NFA $\Rightarrow$

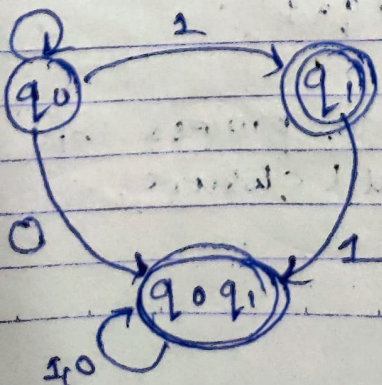
	0	1
q_0	$q_0 q_1$	q_1
q_1	-	$q_0 q_1$

D.F.A T.T. $\Rightarrow$

	0	1
q_0	$q_0 q_1$	q_1
q_1	ϕ	$\{q_1, q_0\}$
$q_0 q_1$	$q_0 q_1$	$q_0 q_1$



Final DFA.

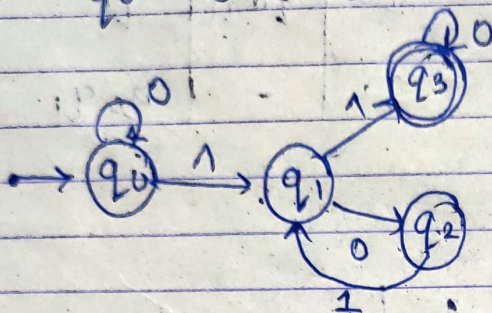


Q. Explain with any 1 e.g. how to convert epsilon NFA to equivalent NFA.

⇒ Eg.

consider fol. epsilon NFA,

q_0 - is start state.



$$Q = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{0, 1\}$$

$$F = \{q_3\}$$

Step 1:- find null closure of all states.

State	Set
q_0	$\{q_0, q_1, q_3\}$
q_1	$\{q_1, q_3\}$
q_2	$\{q_2\}$
q_3	$\{q_3\}$

Step 2:- follow process of null closure, up, null closure

State

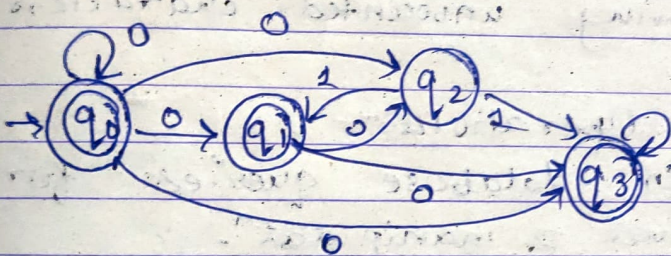
0

1

 q_0 $\{q_0, q_1, q_2, q_3\}$ q_1 $\{q_2, q_3\}$ q_2 $\{q_1, q_3\}$ q_3 $\{q_3\}$

Step 3: Result

Make NFA through transitions obtained in previous state

9.4 Applicatⁿ of Regular expression

1) Text Search & Manipulation:

Used in Text Editor, Programm^g languages, command-line tools for searching, replacing & manipulating the text based on patterns.

2) Data Validation:-

validate inputs in forms or application to ensure it meets specific criteria like email addresses, phone numbers, or passwords.

3) Web scraping:-

Extract specific inf from web browsers by matching pattern which allows us to scrape & parse data efficiently.

4) Programming:-

Integral in programs for tasks like parsing log files, extracting inf, validate data.

5) Data Cleaning:-

useful for cleaning & transforming data, removing unwanted characters.

6) Database Operations:-

employed in database queries for data retrieval & manipulation.

7) Security

role in intrusion detection sys, firewalls, to identify potential threats.

8) NLP

Used in linguistic analysis & text processing task.

Text Search & Manipulation, Database operation, Data Validation, lexical analysis, Program lang, NLP, Web scraping, Data cleaning, Security.

Kleen Closure & Positive Closure.

- Kleen closure, denoted as A^* , $\rightarrow$ set of all possible concatenations of zero or elements from set A .
- Positive clo, $\rightarrow A^+$ $\rightarrow$ it is similar but excludes zero elements. i.e. it includes only non-empty combinations.

So, A^* & A^+ are not equal,
 But there is one case, where they can be equal.

A^* & A^+ will be equal if set A contains at least one element.

or.

When there is at least 1 element, symbol, or one letter, both A^* & A^+ will include all possible combinations of those elements.

$$A = \{a, b\}$$

$$A^* = (\epsilon, ab, ba, aa, bb, \dots)$$

$$A^+ = ab, ba, aa, bb, \dots$$

write Regular exp.

1) Accept all strings start & end with a
& any combinatⁿ of b's betⁿ

$$\Rightarrow R = ab^*b$$

2) The lang start with a but not have consecutive b's.

$$\Rightarrow L = \{a, aba, aab, abaa, aaaa, abab, \dots\}$$

$$\text{Reg. exp} \rightarrow R = \{a + ab\}^*$$

Q.3

Construct FA

$$(11+00)^* \quad (0+1(01))^*$$

$$\text{Q.6. } L = \{aba, ababa, abbba, ababba, \dots\}$$

$$R = \cancel{ab+ab^2}^* ab^*b$$

$$L = \{a, aba, abaa, aab, abab, \dots\}$$

$$L = \{a + ab\}^*$$

$$(11+1)^*$$

Q4. Prove

$(1 + 00^*1) + (1 + 00^*1)^*(0 + 10^* + 1)$ is equal to $0^*1(0 + 10^*1)^*$

$$\begin{aligned}
 \Rightarrow \text{L.H.S} &= (1 + 00^*1) + (1 + 00^*1)^*(0 + 10^* + 1) \\
 &= (1 + 00^*1) [e + (0 + 10^*1)^*(0 + 10^* + 1)] \\
 &\quad \underline{E + R^*R = R^*} \\
 &= (1 + 00^*1) (0 + 10^*1)^* \\
 &= \underline{E} (1 + 00^*1) (0 + 10^*1)^* \\
 &\quad \underline{ER = R} \\
 &= (e + 00^*) 1 (0 + 10^*1)^* \\
 &= 0^*1(0 + 10^*1)^*
 \end{aligned}$$

$$\text{L.H.S} = \text{R.H.S.}$$

Q13) $L = \{a^n b^n \mid n \geq 0\}$ is not regular

$\Rightarrow$ By pump lemma,

Let

$$w = a^n b^n$$

$$|w| = 2n > n$$

Let $w = xyz$ where $|xy| \leq n$

Let $p = 2$

$$xy^2z \neq xyz$$

$$xy^2z = a^{n+2}b^n$$

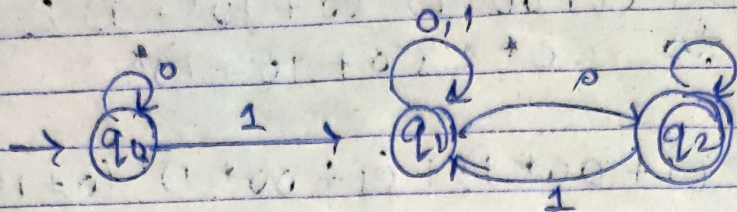
$q \neq 0$, xy^2z is not in form of $a^n b^n$

Thus, xy^2z is not in L .

L is not regular language.

Q 20

NFA to DFA

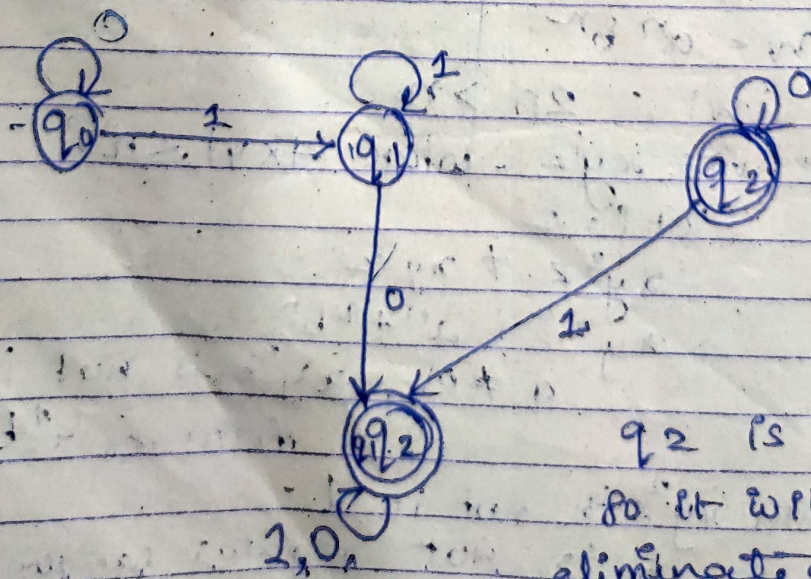


T.T of NFA

	0	1
q ₀	q ₀	q ₁
q ₁	q ₁ q ₂	q ₁
q ₂	q ₂	q ₁ q ₂

T.T of DFA

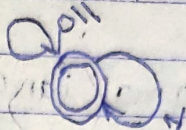
	0	1
q ₀	q ₀	q ₁
q ₁	q ₁ q ₂	q ₁
q ₂	q ₂	q ₁ q ₂
q ₁ q ₂	q ₁ q ₂	q ₁ q ₂



q₂ is unreachable
so it will be
eliminated.

Q18. $L = \{ \underline{011}, 1011, 11011011, \dots \}$

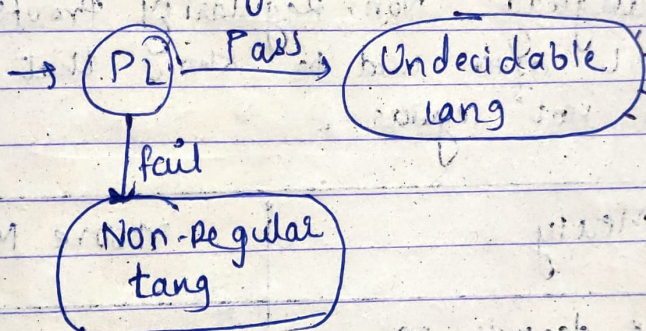
$R = \{ 011 + 1 \}^*$



$011 + 011$

Q17. Pumping lemma? Application

- The language accepted by finite automata is called as Regular language.
- P.L. is used to prove that the language is not regular.
- It is a negative test.



- If L is an infinite lang, such that there exists some +ve integer ' n ' such that any string $w \in L$ has length greater than equal to n ($|w| \geq n$).

→ we can divide it into 3 parts

$w = xyz$

for each $i \geq 0$, $xy^iz \in L$

$|y| \geq 1$

$|xy| \leq n$

$i \geq 0$ $xy^iz \in L$

$|y| \geq 1$

$|xy| \leq n$

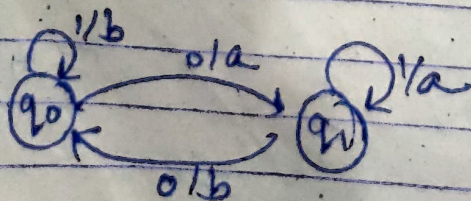
Applications

- **Formal lang. Theory** :-
it is fundamental in understanding hierarchy of formal lang.
- **Network Security** :-
used in security protocols to validate inputs & ensure they conform to expected patterns, which can prevent attacks.
- **Language Non-Regularity Proofs** :-
P.L. often used to show that lang is not regular.

(19)

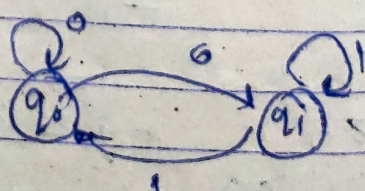
Mealy

- Output depends on present & input state
- generally it has fewer states than moore machine
- The value of 'olp fun' is a fun of transition & changes - when i/p logic on present state is done.



Moore Machine

- Outputs present state
- It has more states
- The value of olp fun is a fun of current state & changes at clock edges whenever ^{state} change occur.



Mealy

- ✓ Output depends on the current state and input state.
- ✓ Typically more complex due to input dependency.
- ✓ Generally requires fewer states.
- ✓ Faster response to inputs.
- ✓ eg: Protocol design where output needs to react quickly

Moore

- ✓ Output depends only on the current state.
- ✓ Typically simpler as o/p is state-dependent.
- ✓ May require more states.
- ✓ Slower response to inputs.
- ✓ eg: Control systems where stable output is crucial.

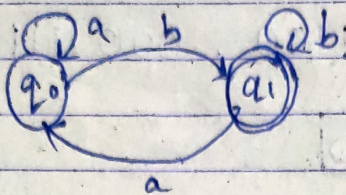
5-star

Finite Automata

PAGE NO.

DATE:

for eg

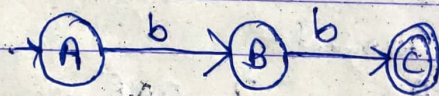


i) $w = \underline{a} \underline{a} \underline{b} \underline{b} \underline{a} \underline{b} \rightarrow \text{accepted}$.

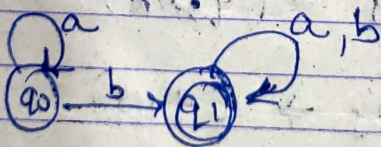
$q_0 q_0 q_1 q_1 q_0 q_1$

$(q_0, aabbab) \vdash (q_0, abbab) + (q_1, bab) \vdash$
 $(q_1, bab) + (q_0, ab) \vdash q_1, b + (q_1) \in F$
 $\underline{\underline{=}}$

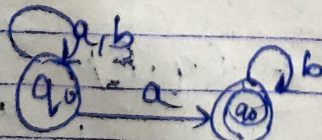
i) ba^*b



$a^*b(a+b)^*$

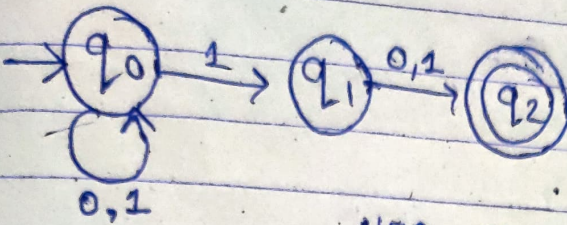


$(ab)^*ab^*$



Finite Aut
Construct

NFA $\rightarrow$ DFA



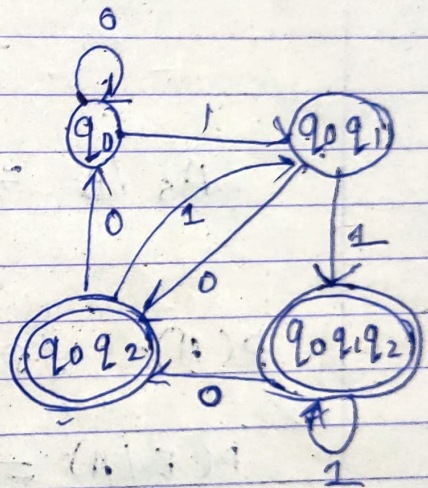
NFA Transition Table

	0	1
q_0	q_0	$q_0 q_1$
q_1	q_2	q_2
q_2	-	-

NOW, DFA T-T

$\Downarrow$

States	0	1
$\rightarrow q_0$	q_0	$q_0 q_1$
$q_0 q_1$	$q_0 q_2$	$q_0 q_1 q_2$
$q_0 q_2$	q_0	$q_0 q_1$
$q_0 q_1 q_2$	$q_0 q_2$	$q_0 q_1 q_2$



$$n \rightarrow 2^n$$

$$3 \rightarrow 2^3 = 8$$

string '00101'

grammar $\rightarrow$

$$S \rightarrow A1B$$

$$A \rightarrow 0A \mid \epsilon$$

$$B \rightarrow 0B \mid B \mid \epsilon$$

LMD:-

$$\begin{aligned} 1) \quad S &\rightarrow A1B \\ &\quad 0A - R_3 \\ &\quad 00A - R_3 \\ &\quad 00 \end{aligned}$$

$$S \rightarrow A1B \rightarrow R_1$$

$$S \rightarrow B \rightarrow R_2$$

$$A \rightarrow 0A \rightarrow R_2$$

$$A \rightarrow \epsilon \rightarrow R_3$$

$$B \rightarrow 0B \rightarrow R_4$$

$$B \rightarrow 1B \rightarrow R_5$$

$$B \rightarrow \epsilon \rightarrow R_6$$

$$1) \quad S \rightarrow A1B$$

$$\rightarrow 0A1B - R_2$$

$$\rightarrow 00A1B - R_2$$

$$\rightarrow 001B - R_3$$

$$\rightarrow 0010B - R_4$$

$$\rightarrow 00101B \rightarrow R_5$$

$$\rightarrow 00101 \rightarrow R_6$$

2) RMD:-

$$S \rightarrow A1B$$

$$\rightarrow A10B - R_4$$

$$A101B - R_5$$

$$A101 \rightarrow R_6$$

$$0A101 \rightarrow R_2$$

$$00A101 \rightarrow R_2$$

$$00101 \rightarrow R_3$$

$$S \rightarrow asb$$

$$S \rightarrow a|b|\epsilon$$

S starts
find name of

$$\textcircled{1} \quad \delta(q, \epsilon, s) = (q, asb)$$

$$\textcircled{2} \quad \delta(q, \epsilon, s) = (q, a) \mid (q, b) \mid (q, \epsilon)$$

$$\textcircled{3} \quad \delta(q, a, a) = (q, \epsilon)$$

$$\textcircled{4} \quad \delta(q, b, b) = (q, \epsilon)$$

Q.7 $S \rightarrow 0s11|so1\epsilon$

$$\delta(q, \epsilon, s) = \{(q, 0s1), (q, 1so), (q, \epsilon)\}$$

$$(q, 0, 0) = (q, \epsilon)$$

$$(q, 1, 1) = (q, \epsilon)$$

Q.8 $S \rightarrow S + S$
 $S \rightarrow id$

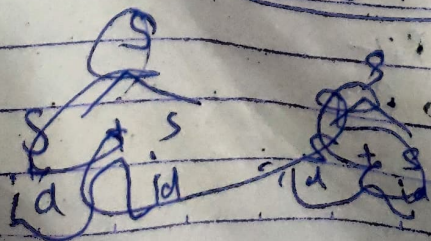
Consider string ~~id + id + id~~ "id + id + id"

$$S \rightarrow S + S$$

$$\rightarrow id + id$$

$$S + S$$

$$id + id$$

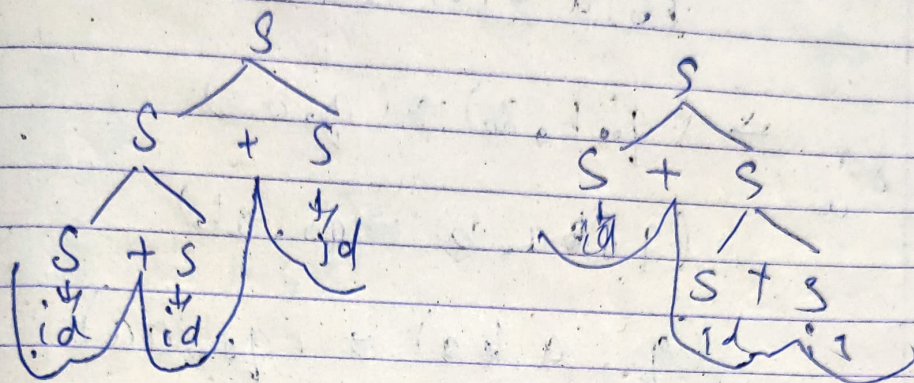


$$S \rightarrow s + s$$

$$\rightarrow s + s + s$$

$$\rightarrow id + id + id$$

Parse



Use left recursion elimin

$$S \rightarrow id \cdot s'$$

$$s' \rightarrow \epsilon \mid +s$$

$$S \rightarrow id \cdot s'$$

$$s' \rightarrow \epsilon \mid +s$$

$$S \rightarrow qsa$$

$$S \rightarrow asa$$

$$S \rightarrow c$$

$$(q, \epsilon, s) = (q, asa)$$

$$(q, \epsilon, s) = (q, asa)$$

$$(q, \epsilon, c) = (q, c)$$

$$(q, a, a) = (q, \epsilon)$$

$$(q, c, c) \rightarrow (q, \epsilon)$$

ncr

ncr p d b r

wcw^r

$\Rightarrow L = (baacaaab, baaacaaab, \dots)$

b|a|a|c|a|a|b

for w : $\Rightarrow (q_0, b, z_0) \rightarrow (q_0, bz_0)$

$(q_0, a, b) \rightarrow (q_0, ab)$

$(q_0, a, a) \rightarrow (q_0, aa)$

a
a
b
<u>z₀</u>

for ϵ :

for w^r : $(q_0, a, a) \Rightarrow (q_{01}, a)$

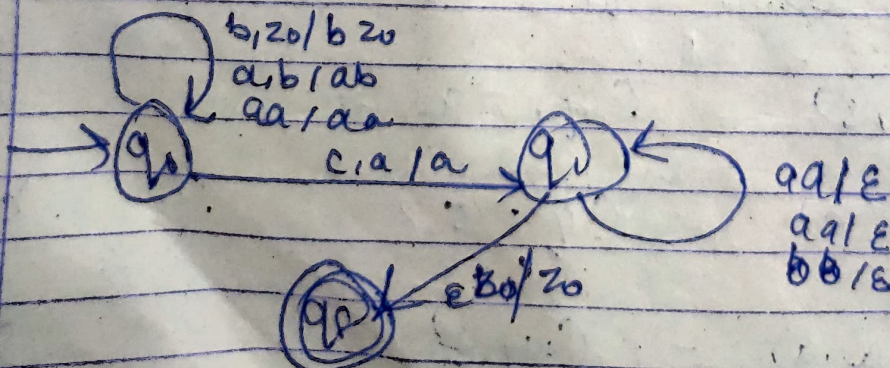
$(q_{01}, a, a) \Rightarrow (q_1, \epsilon)$

$(q_1, a, a) \Rightarrow (q_1, \epsilon)$

$(q_1, b, b) \rightarrow (q_1, \epsilon)$

to f_s

$(q_1, \epsilon, z_0) \rightarrow (q_f, z_0)$



$$S \rightarrow 0S1 \mid 1S0 \mid \epsilon$$

$$\rightarrow (q, \epsilon, S) \Rightarrow \{(q, 0S1), (q, 1S0), (q, \epsilon)\}$$

$$(q, 0, 0) = (q, \epsilon)$$

$$(q, 1, 1) = (q, \epsilon)$$

$$S \rightarrow asb$$

$$S \rightarrow a \mid b \mid \epsilon$$

$$\Rightarrow (q, \epsilon, S) \Rightarrow (q, \epsilon, asb)$$

$$(q, \epsilon, S) \Rightarrow \{(q, a), (q, b), (q, \epsilon)\}$$

$$(q, a, a) = (q, \epsilon)$$

$$(q, b, b) = (q, \epsilon)$$

$$S \rightarrow aAA$$

$$A \rightarrow as \mid bs \mid a$$

$$\begin{aligned} \Rightarrow (q, \epsilon, S) &= (q, aAA) \\ (q, \epsilon, A) &= \{(q, as), (q, bs), (q, a)\} \\ q, aa &= q, \epsilon \\ bb &= q, \epsilon \end{aligned}$$

FIA.

Q B-2

PAGE NO.

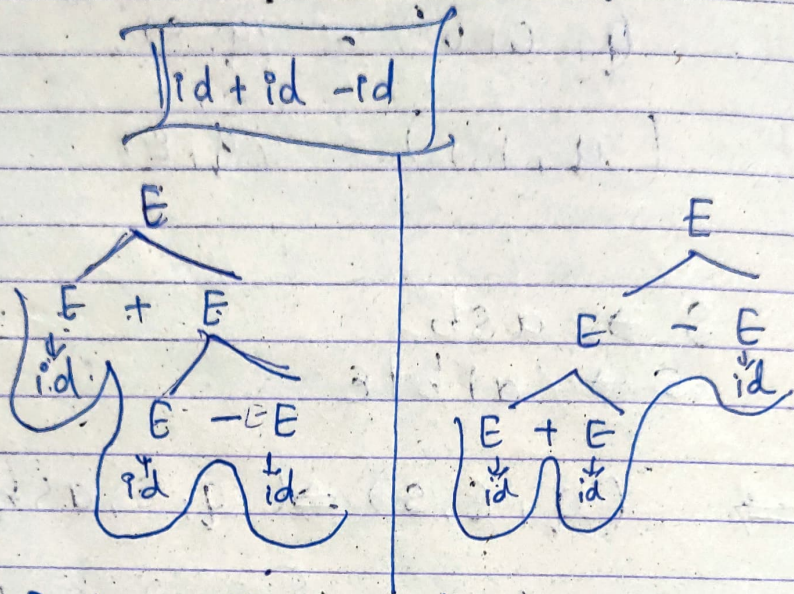
DATE:

Q.

$$E \rightarrow E + E$$

$$E \rightarrow E - E \quad \equiv \quad E \rightarrow E + E \mid E - E \mid id$$

$$E \rightarrow id$$



✓ ∴ It is ambiguous in nature.

Q2

$$S \rightarrow aB \mid bA$$

$$A \rightarrow a \mid aS \mid bAA$$

$$B \rightarrow b \mid bS \mid aBB$$

⇒

string → "aabbabba"

LMD:-

$$S \rightarrow aB$$

$$a a B B - R_1$$

$$a a b S B - R_7$$

$$a a b b A B - R_2$$

$$a a b b a B - R_3$$

$$a a b b a b S - R_4$$

$$a a b b a b b A - R_5$$

$$a a b b a b b a - R_3$$

$$S \rightarrow aB \rightarrow R_1$$

$$S \rightarrow bA \rightarrow R_2$$

$$A \rightarrow a \rightarrow R_3$$

$$A \rightarrow aS \rightarrow R_4$$

$$A \rightarrow bAA \rightarrow R_5$$

$$B \rightarrow b \rightarrow R_6$$

$$B \rightarrow bS \rightarrow R_7$$

$$B \rightarrow aBB \rightarrow R_8$$

RMD: -

$$S \rightarrow aABB \quad \text{---} \quad R_1$$

→ $aABbs$ — R_7

$aabbba$ — R_2

$aaBbba \rightarrow R_3$

$aab\bar{s}bba$ — R7

aabbAbba — R₂

aabbabba - R₃

Q4. Ambiguous or not

$$S \rightarrow a S_b | SS$$

$S \rightarrow E$

for each string "aabb",

$$S \rightarrow a S b \mid R,$$

→ ~~acb~~

→ aaSbb - R1

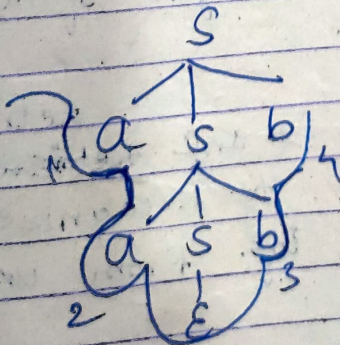
$\rightarrow aa \in bb. - R_3$

$s \rightarrow aabb -$

$$S \rightarrow a s b - R_1$$

$S \rightarrow SS - R_2$

$S \rightarrow E - R_3$



q is ambiguous

Applicatⁿ of Context free Grammar

⇒ used in CS & formal language theory.
Context-free language (CFLs) are a type of formal lang that can be generated by context-free grammar (CFGs).

Applicatⁿ:-

- Programming Language:-
Many pl. are designed to be context-free. for instance, the syntax of lang like C, Java, Python & others are often described using CFL.
- Syntax Analysis & Parsing:-
used in parsing algorithms such as LL parsers, LR parsers, & CYK parsers, which are essential comp of compilers & interpreters.
- NLP:-
used for syntactic analysis of natural language.
- Document Analysis & Inf Retrieval:-
used in D.A & I.R. to recognize & extract structured inf. from unstructured text data.

DNA sequence analysis
 used for analyzing DNA & RNA sequences.
 helps identify patterns & structures
 with genetic data.

Compiler Optimization

Ambiguity Concept with eg

A grammar is said to be ambiguous if there exists more than one leftmost derivation or more than one RMD or more than one parse tree for given input string.

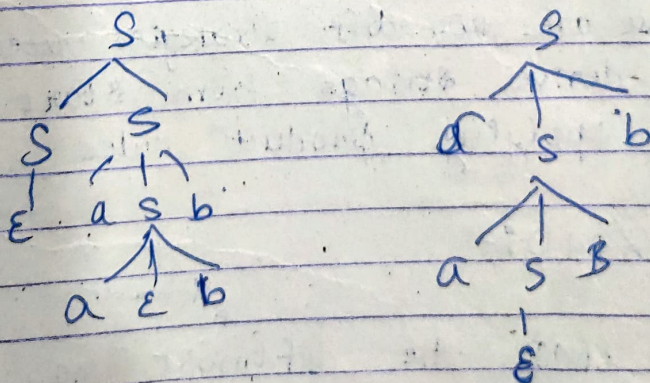
If grammar is not ambiguous, then it is called unambiguous.

eg

$$S = aSb \mid SS$$

$$S = \epsilon$$

for string $aabb$, the above g. generates 2 parse trees.



Since there are 2 parse trees for a single string, we can say grammar is ambiguous grammar.

CFL, stack, powerful,
complex lang, pars^g CFG, HTML parsing

PAGE NO.

DATE:

Pushdown

~~Finite~~ Automata

Finite

~~Pushdown~~ Automata

1) recognizes context-free languages

2) has an additional stack of memory

3) more powerful

4) recognizes more complex languages.

5) capable of parsing CFG.

6) Eg. Parsing arithmetic expressions, HTML parsing

1) recognizes regular languages

2) doesn't have memory beyond its state

3) less powerful

4) recognizes limited to regular languages

5) cannot parse CFG.

6) Lexical analyses in compilers, pattern matching

Q

Define LMD & RMD in detail.

→ These are two diff. strategies used in CFG. to derive strings from start symbol by applying productⁿ rules.

Q LMD:-

In LMD, the ^{rightmost} leftmost non-terminal symbol in current sentential form is replaced with its corresponding production rule.

- This process continues until no non-terminal symbols remain, resulting in desired string

• Process of RMD:-

1. Start with start symbol

Begin with start symbol of CFG as initial sentential form.

2. Replace LM Non-terminal:-

At each step, identify LM NT symbol & replace it with its corresponding production rule.

3. Repeat until no non-terminal left:-

Repeat this process iteratively until no NTS remain.

4. Result:-

The resulting string obtained is the string derived using LMD.

LRMD:-

Same for RMD, but change (left) with (right) - -

PDA tuples:-

$$P = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$$

- 1) Q = finite set of states like FA.
- 2) Σ = Input symbol like FA.
- 3) Γ = stack Alphabet (Push it to stack)
- 4) δ = Transition fund, $\delta = Q \times (\Sigma \cup \epsilon) \times \Gamma$
- 5) q_0 = initial state $\rightarrow Q \times \Gamma$
- 6) z_0 = stack start symbol
- 7) F = final state

$$Q, \Sigma, \Gamma, \delta, q_0, z_0, F$$